NOVA IMS INFORMATION MANAGEMENT SCHOOL

What a for can change

mutable lists, immutable strings

Computation I · Lecture 07

Karina Brotto Rebuli
krebuli@novaims.unl.pt

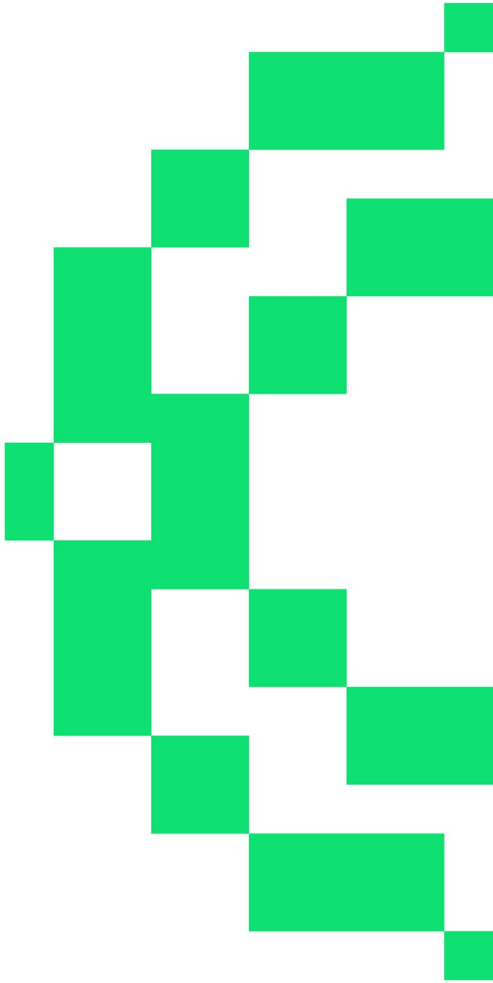
The list that would not change

Exercise 9, replayed — double every score

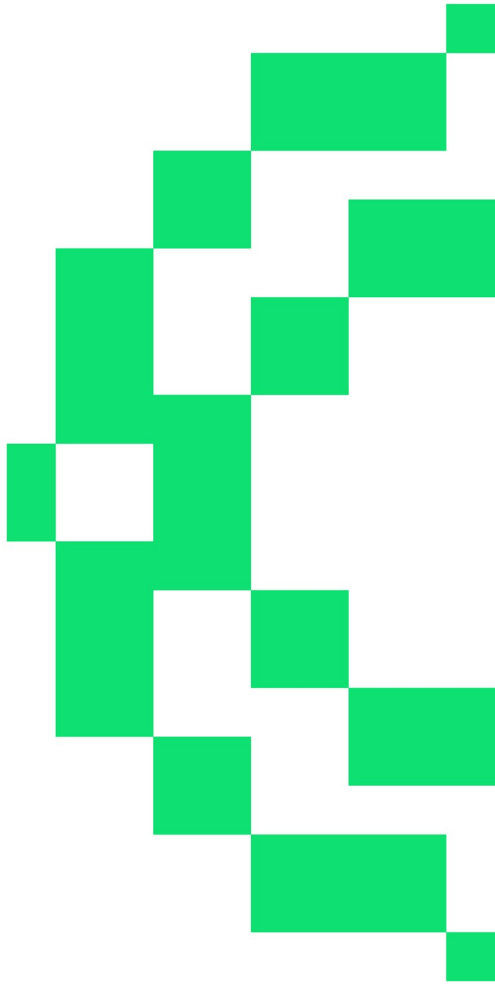
```
scores = [10, 12, 15]
for x in scores:
    x = x * 2
print(scores)
```

Two habits, one gap

Last week a loop read a list and walked a word. But doubling every score inside the loop changed nothing — it ran and lied — and reversing a word took three numbers and an index expression. Today one slice turns everything around, the list really changes — and Saturday is coming.



```
"""A list can be modified inside a for loop."""
```



John McCarthy

1958 — A sequence that changes shape as the program runs.



Image source

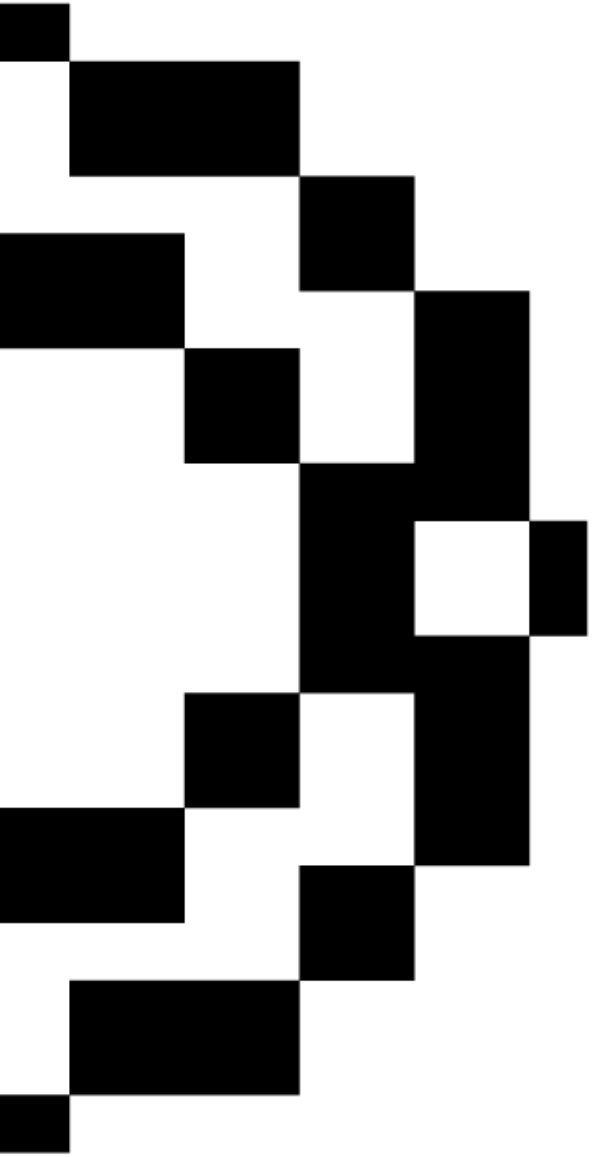
The problem: he needed to hold a sequence whose length he did not know in advance — one a program could grow, take apart, and put back together while it ran. The arrays of the day were fixed the moment they were made.

Three questions today

A slice is always a new object.

Grow a list, and change what is inside it.

Get ready for Saturday.



01.

Take it apart

what a slice hands back is always new

Backwards, the hard way and in one move

Exercise 10, replayed — the word, last to first

`for` loop

```
word = "grade"
for i in range(len(word) - 1, -1, -1):
    print(word[i])
```

START (INCLUDED)	len(word) – 1 ∴ i = 4
STOP (EXCLUDED)	-1 ∴ until i = 0
STEP	-1 ∴ backwards

slicing

```
word = input("Word: ")
print(word[::-1])
```

START	last position ∴ i = 4
STOP	first position ∴ i = 0
STEP	-1 ∴ backwards

A slice can skip

s[start:stop:step]

```
s = "abcdefghijkl"
print(s[0:len(s):2])
print(s[0:len(s):3])
```

```
>>> acegik
>>> adgj
```

LIVE

A slice is a new sequence

The original is untouched

A LIST:

```
nums = [1, 2, 3, 4]
half = nums[::2]
half[0] = 99
print(half)
print(nums)
```

```
>>> [99, 3]
>>> [1, 2, 3, 4]
```

A STRING:

```
word = "computation"
part = word[::2]
part = part.replace("m", "M")
print(part)
print(word)
```

```
>>> cMuain
>>> computation
```

Slicing, the whole rule

`start` in, `stop` out, `step` jumps — and what comes back is always a new object.

`start`

WHAT IT MEANS

Where the slice begins — included.

If you leave it out

`0` — the very beginning.

`stop`

WHAT IT MEANS

Where it ends — excluded, never reached.

If you leave it out

`len(s)` — all the way to the end.

`step`

WHAT IT MEANS

How far and the direction to each move jumps.

If you leave it out

`+1` — every element, left to right.

OUT OF RANGE, THE TWO PART WAYS

A slice never crashes — `s[2:99]` simply stops at the end. An index past the end still does: `s[99]` raises `IndexError`.

02.

The list that grows and changes

Build it as the loop runs

LIVE

Why the first version lied

x is a copy of the value, not the slot in the list

```
scores = [10, 12, 15]
for x in scores:
    x = x * 2
    print(x)
print(scores)
```

the list

a copy of each value

modifies the copy

the list, unchanged

```
>>> 20
>>> 24
>>> 30
>>> [10, 12, 15]
```

LIVE

Now the list really changes

`range(len(L))` hands you each position — the assignment lands in the slot, not in a copy

```
scores = [10, 12, 15]
for i in range(len(scores)):
    scores[i] = scores[i] * 2
print(scores)
```

the positions: 0, 1, 2

the slot itself

the list, changed

```
>>> [20, 24, 30]
```

LIVE

A list can start empty

append adds one item each turn

```
squares = []
for i in range(1, 6):
    squares.append(i * i)
print(squares)
```

```
>>> [1, 4, 9, 16, 25]
```

LIVE

Remove from the list you are walking

`remove` shifts what is left — walk a copy, change the original

THE TRAP:

```
grades = [12, 3, 2, 18, 15]
for g in grades:
    if g < 10:
        grades.remove(g)
print(grades)
```

```
>>> [12, 2, 18, 15]
```

LIVE

Remove from the list you are walking

`remove` shifts what is left — walk a copy, change the original

THE TRAP:

```
grades = [12, 3, 2, 18, 15]
for g in grades:
    if g < 10:
        grades.remove(g)
print(grades)
```

```
>>> [12, 2, 18, 15]
```

ON A COPY:

```
grades = [12, 3, 2, 18, 15]
for g in grades[:]:
    if g < 10:
        grades.remove(g)
print(grades)
```

```
>>> [12, 18, 15]
```

LIVE

A new string, letter by letter

the string never changes — you grow a new one from it

```
word = "grade"
new_word = ""
for letter in word:
    new_word = new_word + letter + "!"
print(new_word)
```

```
>>> g!r!a!d!e!
```

Count and sum only the ones that match

A counter is an accumulator too

```
values = [3, "12", 5, "20", 8, 15, 10]
count = 0
total_up_to_10 = 0
for v in values:
    v = int(v)
    if v <= 10:
        count += 1
        total_up_to_10 += v
print(count, total_up_to_10)
```

```
>>> 4 26
```

LIVE

Fix the ones that break the rule

Clamp every grade between 0 and 20

TWO SEPARATE CHECKS:

```
grades = [12, 25, -3, 30, 9]
for i in range(len(grades)):
    if grades[i] > 20:
        grades[i] = 20
    if grades[i] < 0:
        grades[i] = 0
print(grades)
```

ONE CHECK, TWO BRANCHES:

```
grades = [12, 25, -3, 30, 9]
for i in range(len(grades)):
    if grades[i] > 20:
        grades[i] = 20
    elif grades[i] < 0:
        grades[i] = 0
print(grades)
```

```
>>> [12, 20, 0, 20, 9]
```

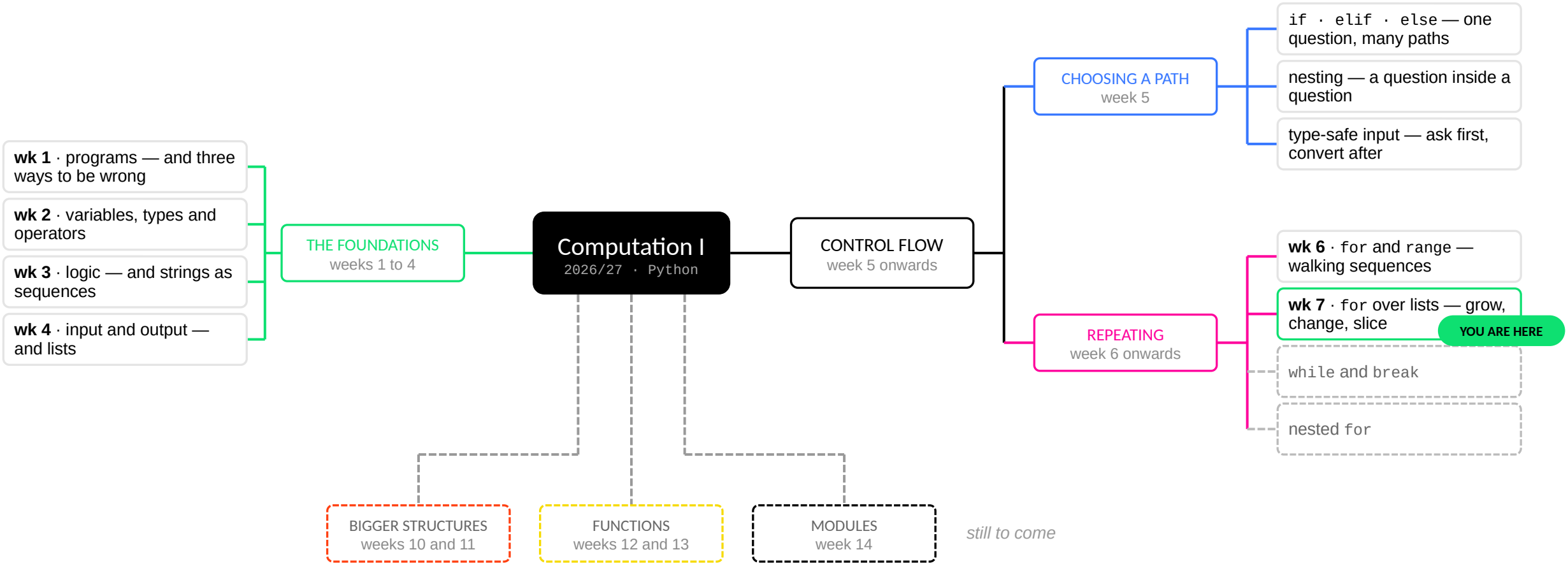
03.

Before Saturday

Everything from week 1 to week 7

The course so far

The solid part is on Saturday's test — the dashed part is still to come



Review 1 • Your age on test day

Write a program that asks for the students' birthday dates — any number of them — in one line, as DD/MM/YYYY and comma-separated; assume the dates are typed in the expected format. Print each student's age on test day, 24/10/2026, and end with how many will be 18 or over. One run, for example:

```
Write the birthday dates (DD/MM/YYYY): 30/10/2007, 24/10/2008, 05/03/2009, 12/01/2005

On test day, student 1 will be 18 (birthday 30/10/2007)
On test day, student 2 will be 18 (birthday 24/10/2008)
On test day, student 3 will be 17 (birthday 05/03/2009)
On test day, student 4 will be 21 (birthday 12/01/2005)
3 students will be 18 or over on test day
```

Review 2 · Fix the ride check

This program should read the heights, in cm, typed in one line, comma-separated; print one line for each person tall enough to ride — 140 cm or more — and end with how many can ride. Run with 152,98,170 it crashes on line 3; fixed, it lets the 98 ride and counts zero. Fix every line to blame.

```
line = print(input("Heights: "))
riders = 0
for h in line.split(", "):
    if h >= "140":
        print(f"{h} cm: can ride")
        riders + 1
print(f"{riders} can ride")
```

Review 3 · Complete the stock count

Complete the program: the products and their quantities arrive in one string, as name:quantity pairs separated by semicolons. It prints a restock alert for every product at zero, then the total of pieces — here, banana: RESTOCK and then 19 pieces in stock. Write what goes in <A>, <B>, <C> and <D>.

```
stock = "apple:12;banana:0;kiwi:7"
total = 0
for item in <A>:
    parts = <B>
    total = <C>
    if <D>:
        print(parts[0] + ": RESTOCK")
print(f"{total} pieces in stock")
```

Review 4 · Predict what it prints

Read the program below and write exactly what it prints — five lines, in order. On paper, before any run. Then run it and compare. Everything in it was taught by week 5.

```
code = " NOVA-2026 "
sizes = [38, 41, 35, 44]
tag = code.strip()[:4].lower()
print(tag * 2, len(tag))
print(code.strip()[::-1])
print(sizes[1::2])
sizes.remove(41)
print(sizes, 41 in sizes)
print(sizes[-1] // 10, sizes[-1] % 10)
```

Three questions, answered

In one sentence each

Take it apart
start in, stop out, step jumps; s[::-1] reverses — and what comes back is always a new object; the original never changes

Grow and change
start empty and append one item per turn; to change what is there, write to L[i] — the slot, not a copy

Get ready for Saturday
seven weeks of material, on paper first.

Before the next theoretical lecture

On your own

Think Python — Sections 9.1 A list is a sequence · 9.2 Lists are mutable · 9.3 List slices · 9.4 List operations · 9.5 List methods · 9.6 Lists and strings · 9.7 Looping through a list · 9.13 Debugging.

Coding test 1 — on Saturday. It covers everything up to and including the for loop — weeks 1 to 7.

One habit — on paper first, run second: that is exactly how you study to Saturday's coding test.

Exercise 1 · Predict what it prints

Read the program below and write exactly the list it prints. On paper, before any run. Then run it and compare.

```
result = []
for i in range(4):
    result.append(i * 10)
print(result)
```

Exercise 2 · Predict what it prints

Read the program below and write exactly the number it prints. On paper, before any run. Then run it and compare.

```
nums = [4, 8, 15, 16, 23, 42]
total = 0
for n in nums:
    total = total + n
print(total)
```

Exercise 3 · How many are 18 or over

Write a loop that counts how many of the ages are 18 or over, then print the count — one number, exactly as the output below shows. Start from the list given in the panel. Use this lecture's counter, with week 5's if choosing what counts.

```
ages = [12, 20, 17, 34, 8, 19]
# your loop here
```

```
>>> 3
```

Exercise 4 · Fix the program

This program should double every price in the list. It runs and raises nothing, yet the print shows the list unchanged. Find the line to blame, say what it computes and where that value goes, and write the fix.

```
prices = [10, 20, 30]
for i in range(len(prices)):
    prices[i] * 2
print(prices)
```

Exercise 5 · One degree warmer

Write a loop that adds one degree to every temperature, then print the list — exactly as the output below shows. Start from the list given in the panel. Use this lecture's mutation by index: reach the slot, not the copy.

```
temps = [15, 18, 21]
# your loop here
```

```
>>> [16, 19, 22]
```

Exercise 6 · Every low grade to zero

Write a loop that sets every grade below 10 to zero, then print the list — exactly as the output below shows. Start from the list in the panel. Use this lecture's mutation by index, with week 5's if choosing which slots change.

```
grades = [8, 14, 20, 5, 17]
# your loop here
```

```
>>> [0, 14, 20, 0, 17]
```

Exercise 7 · Predict what it prints

Read the two prints below — the step in a slice, this lecture's material. On paper, before any run: write both lines exactly. Then run it and compare.

```
s = "programming"
print(s[::2])
print(s[::-1])
```

Exercise 8 · Predict what it prints

Read the two prints below — a slice of the list, then the whole list reversed. On paper: write both lines exactly, brackets included. Then run it and compare.

```
nums = [10, 20, 30, 40, 50]
print(nums[1:4])
print(nums[::-1])
```

Exercise 9 · Every name capitalized

Write a loop that turns every element into a capitalized string, then print the list — exactly as the output below shows. The names arrived mixed with numbers, so convert each element with `str` (week 2) before `capitalize` (week 3). Use this lecture's mutation by index.

```
names = ["ana", 12, "bruno", 7, "carla"]
# your loop here
```

```
>>> ['Ana', '12', 'Bruno', '7', 'Carla']
```

Recap 1 · Predict what it prints

Read the program below and write exactly what it prints — every line, in order. On paper, before any run. Then run it and compare.

```
nums = [3, 8, 5, 2]
total = 0
for i in range(len(nums)):
    if nums[i] > 4:
        nums[i] = nums[i] - 4
    total = total + nums[i]
print(nums, total)
print(nums[::-1][0])
```

Recap 2 · Choose the fragment

The program should replace every negative number in values by 0, changing the list in place. It starts with values = [4, -1, 7, -3] and ends with print(values). Exactly one of the four fragments below does the job. Choose it — and say what goes wrong and propose a fix in each of the other three.

```
# A
for i in range(len(values)):
    if values[i] < 0:
        values[i] = 0
# B
for v in values:
    if v < 0:
        v = 0
```

```
# C
for i in values:
    if values[i] < 0:
        values[i] = 0
# D
for i in range(len(values)):
    if i < 0:
        values[i] = 0
```

NOVA IMS INFORMATION MANAGEMENT SCHOOL

```
input("Questions? ")
```

Acreditações e Certificações da NOVA IMS

eduniversal · QS STARS · ABET Computing Accreditation Commission · iSchools · EMOS · UNIGIS · USGIF · Erasmus Mundus Programme Education and Culture DG · Lapcer · IQNet

Cofinanciado por

Portugal 2030 · Lisboa 2030 · Co-funded by the European Union · fct Fundação para a Ciência e a Tecnologia · PRR Plano de Recuperação e Resiliência · República Portuguesa · Financiado pela União Europeia NextGenerationEU

© Karina Rebuli, 2026

Exercise 10 · Give the player three tries ★

Run the game in the panel — the player gets exactly three guesses. Play it twice: once guessing right, once never guessing right. Then describe what the program cannot do, with what has been taught so far.

```
secret = 7
for attempt in range(3):
    guess = int(input("Guess: "))
    if guess == secret:
        print("You got it!")
```

Exercise 11 · Report where the first negative is ★

Run the program in the panel — it should report where the first negative number is. Read its output closely. Then try, with only what has been taught so far, to make it report the first one and nothing more.

```
values = [4, 7, -2, 9, -5]
for i in range(len(values)):
    if values[i] < 0:
        print("Negative at", i)
```